



Tecnológico Nacional de México

Campus Querétaro

Entregable 11,
titulado **Entregable 11 Unidad 1**

Producto de la asignatura

Ingeniería de software

Facilitado por:

Julio Alejandro Villeda Maldonado

Desarrollado por:

Brayan Eduardo Suarez Cruz



Verificación de Coherencia con E10 (Comportamiento vs. Estructura)

Para garantizar que la estructura soporte el comportamiento modelado anteriormente, se han verificado los siguientes puntos:

- **Trazabilidad de Métodos:** Cada mensaje enviado a un objeto en el diagrama de secuencia (E10) se ha traducido en una operación pública dentro de las clases de la capa Controller, Service o Repository en E11.
- **Persistencia:** Se confirma que por cada flujo que requería guardar información en E10, existe un método guardar() o actualizar() en la clase Repository correspondiente.
- **Validaciones:** Las decisiones lógicas identificadas en los fragmentos alt de E10 están mapeadas como métodos booleanos o de resultado en la capa Service (ej. validarDatos(), validarDisponibilidad()).

Identificación de Clases Críticas

Se han identificado las siguientes clases como puntos neurálgicos del sistema debido a su carga de responsabilidad o volumen de relaciones:

- **Entidades Centrales (Entity):** Clases como Proyecto y Transacción son críticas ya que representan la información persistente fundamental y poseen múltiples asociaciones con otras entidades.
- **Servicios de Negocio (Service):** PresupuestoService y TransaccionService son críticos porque centralizan las reglas de negocio y las validaciones complejas; un error aquí compromete la integridad de los datos.
- **Repositorios (Repository):** TransaccionRepository es vital para el rendimiento, dado que manejará el mayor volumen de operaciones de lectura/escritura.

Análisis de Acoplamiento y Dependencias

El sistema implementa un Bajo Acoplamiento siguiendo la dirección de dependencia obligatoria:

- **Direccionalidad:** Se respeta estrictamente el flujo Controller → Service → Repository → Entity. Se ha verificado que ninguna Entity dependa de un Controller, evitando dependencias circulares.
- **Independencia de Capas:** Los controladores no conocen la implementación de la base de datos, solo invocan interfaces de servicios, lo que facilita el mantenimiento y cambios tecnológicos futuros.

Análisis de Cohesión



Se ha evaluado la Alta Cohesión en las clases diseñadas:

- Responsabilidad Única: Cada controlador está limitado a un módulo específico (ej. ReservaController no gestiona usuarios), evitando la creación de "Clases Dios" o SistemaController.
- Especialización: Las entidades contienen únicamente atributos del dominio y no lógica de persistencia o de negocio compleja, delegando estas funciones a sus capas respectivas.